

Лабораторная работа №27

Разработка сетевых приложений для двусторонней связи в реальном времени

1 Цель работы

1.1 Изучить процесс разработки сетевых приложений с использованием SignalR на C#;

2 Литература

2.1 Основы SignalR // Metanit.com – URL: <https://metanit.com/sharp/signalr/1.12.php>. – Режим доступа: свободный. – Текст : электронный.

3 Подготовка к работе

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Создание сервера для чата на SignalR

5.1.1 Создать новый проект ASP.Net minimal api.

5.1.2 Подключить в файле Program пакет Microsoft.AspNetCore.SignalR

5.1.3 Зарегистрировать в сервисах SignalR

```
builder.Services.AddSignalR();
```

5.1.4 Создать класс ChatHub – наследник класса Hub, в котором реализовать асинхронный метод для ретрансляции сообщений, полученных от пользователя другим пользователям с помощью вызова удаленного callback-метода.

```
await Clients.All.SendAsync(имя_callback, отправляемые_данные);
```

5.1.5 Создать endpoint для ChatHub при помощи

```
app.MapHub<ChatHub>("/адрес");
```

5.2 Создание клиента для чата на SignalR

5.2.1 Создать новый проект настольного приложения

5.2.2 При запуске приложения пользователь должен указать свое имя, после чего ему становится доступна страница для получения и отправки сообщений.

5.2.3 Установить в проекте пакет Microsoft.AspNetCore.SignalR.Client

5.2.4 Установить подключение к серверу SignalR при помощи

```
var connection = new HubConnectionBuilder()  
    .WithUrl("адрес hub")  
    .Build();
```

5.2.5 Настроить callback для получения сообщения от сервера и

отображения его в списке на странице исходя из данных, отправляемых в п.5.1.4

```
connection.On<типы_получаемых_данных>(имя_callback,  
(получаемые_данные) =>  
{  
    //Код для вывода полученного сообщения  
});
```

В зависимости от реализации может потребоваться использовать Dispatcher.Invoke(() =>...) для работы с UI из другого потока

5.2.6 Запустить соединение к серверу

```
await connection.StartAsync();
```

5.2.7 Добавить в приложение поле ввода и кнопку для отправки сообщения. При нажатии на кнопку отправлять их на сервер при помощи

```
await connection.InvokeAsync(имя_метода_обработки_сообщений,  
отправляемые_данные);
```

5.3 Реализация комнат-чатов

5.3.1 Добавить в класс ChatHub два поля типа ConcurrentDictionary<string, string> для хранения данных о именах пользователей и подключениях к комнатам.

5.3.2 Реализовать метод JoinRoom(string roomName, string userName), в котором необходимо сохранить данные пользователя в соответствующие словари

```
// Получаем соединение  
var connectionId = Context.ConnectionId;  
  
// Привязать соединение к комнате и пользователю  
ConnectionToRoom[connectionId] = roomName;  
ConnectionToUser[connectionId] = userName;  
// Добавляем соединение в группу по имени комнаты  
await Groups.AddToGroupAsync(connectionId, roomName);
```

5.3.3 Далее при обработке сообщений на сервере можно использовать

```
// Проверка наличия имени пользователя и комнаты в  
ConcurrentDictionary  
if (ConnectionToRoom.TryGetValue(connectionId, out var room) &&  
    ConnectionToUser.TryGetValue(connectionId, out var user))  
{  
    // Отправка данных клиентам в определенной группе  
    await Clients.Group(room).SendAsync("ReceiveMessage", user,  
message);  
}
```

5.3.4 Необходимо обработать отключение пользователя от группы при разрыве соединения с клиентом, для этого необходимо переопределить стандартный метод OnDisconnectedAsync и использовать в нем

```
await Groups.RemoveFromGroupAsync(connectionId, room);
```

5.3.5 Реализовать для пользователя ввод названия комнаты при запуске приложения.

6 Порядок выполнения работы

- 6.1 Запустить MS Visual Studio и создать оконное приложение C#.
- 6.2 Выполнить все задания из п.5 в одном решении.
- 6.3 Ответить на контрольные вопросы.

7 Содержание отчета

- 7.1 Титульный лист
- 7.2 Цель работы
- 7.3 Ответы на контрольные вопросы
- 7.4 Вывод

8 Контрольные вопросы

- 8.1 Что такое SignalR?
- 8.2 Каким образом осуществляется обмен данными между клиентом и сервером SignalR?

9 Приложение

SignalR Core представляет библиотеку от компании Microsoft, которая работает поверх ASP.NET и которая предназначена для создания приложений, работающих в режиме реального времени. В частности, ее можно использовать вместе с ASP.NET Core. SignalR использует двунаправленную связь для обмена сообщениями между клиентом и сервером, благодаря чему сервер может отправлять в режиме реального времени всем клиентам некоторые данные, а клиенты также в режиме реального времени могут передавать данные серверу и другим клиентам.

Создание простейшего сервера

Определим в проекте следующий простейший класс хаба:

```
using Microsoft.AspNetCore.SignalR;  
  
namespace SignalRApp  
{  
    public class ChatHub : Hub  
    {  
        public async Task Send(string message)  
        {  
            await this.Clients.All.SendAsync("Receive", message);  
        }  
    }  
}
```

В файле **Program.cs** определим следующий код:

```
using SignalRApp; // пространство имен класса ChatHub  
  
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddSignalR();  
  
var app = builder.Build();
```

```
app.UseDefaultFiles();
app.UseStaticFiles();

app.MapHub<ChatHub>("/chat");
app.Run();
```

Создание простейшего клиента

В файле **Program.cs** определим следующий код:

```
var connection = new HubConnectionBuilder()
    .WithUrl("http://localhost:5000/chat")
    .WithAutomaticReconnect()
    .Build();

// обработка входящих сообщений от сервера
connection.On<string>("ReceiveMessage", (message) =>
{
    Console.WriteLine(message);
});
await connection.StartAsync();
Console.WriteLine("Connected!");

while (true)
{
    var message = Console.ReadLine();
    if (string.IsNullOrEmpty(message))
        break;
    await connection.InvokeAsync("SendMessage", message);
}

await connection.StopAsync();
```

Группы в SignalR представляют коллекцию подключений, которые ассоциированы с именем группы. SignalR позволяет добавлять и удалять клиентов из группы. Один клиент может входить сразу в несколько групп.

Для работы с группами в классе Hub определено свойство **Groups**, которое представляет объект **IGroupManager**. Он имеет два метода:

- **AddToGroupAsync(connectionId, groupName)**: добавляет клиента с идентификатором connectionId в группу с именем groupName
- **RemoveFromGroupAsync(connectionId, groupName)**: удаляет клиента с идентификатором connectionId из группы

Также у объекта IHubCallerClients определено ряд методов, которые позволяют выбрать определенные группы для отправки им сообщения. Для вызова метода у клиентов в группах можно использовать ряд методов:

- **Group(string groupName)**: вызывает метод у клиентов определенной группы
- **GroupExcept(string groupName, IReadOnlyList<string> connectionIds)**: вызывает метод у клиентов группы по имени groupName

за исключением тех клиентов, id которых передаются в качестве второго параметра

- **Groups(IReadOnlyList<string> groupNames):** вызывает метод у клиентов групп, названия которых передаются в метод
- **OthersInGroup(string OthersInGroup):** вызывает метод у клиентов определенной группы за исключением текущего клиента